

Análisis de Algoritmos
Fuerza Bruta y BackTracking

Alejandro Cifuentes Arroyave, Juan Daniel Martínez Fernández, Samuel Posada
Londoño

EAFIT
Análisis y Diseño de Algoritmos
Carlos Alberto Álvarez Henao
30 de agosto de 2026

1. Introducción.

El presente informe documenta el diseño, desarrollo, validación y análisis experimental de dos algoritmos que son Fuerza Bruta. El objetivo es determinar los límites prácticos de arquitecturas de hardware estándar frente a problemas de explosión combinatoria.

2. Contexto del Problema.

Esta práctica introduce los paradigmas de Fuerza Bruta y Backtracking a partir de un dominio común: contraseñas y espacios de cadenas sobre un alfabeto. En el Módulo FB, el estudiante asume el rol de analista de seguridad y mide experimentalmente el tiempo requerido para hallar una contraseña probando todas las combinaciones posibles. En el Módulo BT, asume el rol de diseñador de políticas y debe generar o contar las contraseñas que cumplen una política de seguridad, sin recorrer primero todo el espacio de cadenas. Esta estructura busca que el estudiante contraste directamente la exploración exhaustiva sin poda propia de Fuerza Bruta con la exploración incremental de Backtracking, que descarta ramas no viables tan pronto se identifican.

3. Fundamentación teórica.

Fuerza Bruta y Hash.

La fuerza bruta genera de forma secuencial todas las cadenas posibles (s) y calcula $H(s) = SHA - 256(s)$ hasta encontrar un hash coincidente. Dado un Alfabeto Σ la dimensión del espacio de búsqueda es $|S| = |\Sigma|^n$

Backtracking y Poda.

El Backtracking modela el espacio de búsqueda como un árbol de decisión. Cada nivel del árbol representa la asignación del carácter en la posición i . La función de factibilidad evalúa si la solución actual puede derivar en una solución válida. Si no es así, la rama se poda completamente evitando explorar nodos descendientes.

4. Modelamiento del Espacio de Búsqueda.

Fuerza Bruta: Espacio sin restricciones. $S = \Sigma^n$

- Alfabeto A1 (a-z): $n = 4$ (456.976), $n = 5$ (11.88M), $n = 6$ (308,91 M)
- Alfabeto A2 (a-z) y (0-9): $n = 4$ (1.67 M), $n = 5$ (60.44M)

Backtracking: Espacio restringido sobre un alfabeto de 67 caracteres (26 minúsculas, 26 mayúsculas, 10 números, 5 símbolos: !@#\$%).

- **Política (Semilla 2472):** ≥ 2 minúsculas, ≥ 1 mayúscula, ≥ 1 número, ≥ 1 símbolo y sin caracteres iguales consecutivos.
- **Factibilidad:** La suma de mínimos ($2 + 1 + 1 + 1 = 5$) es $\leq n$ para $n \in \{6, 8, 10\}$, garantizando la validez del espacio.

5. Diseño Algorítmico.

Módulo de Fuerza Bruta

- **Búsqueda por Fuerza Bruta:** Recorre el espacio de forma iterativa simulando un contador en base al tamaño del alfabeto. Usa un vector de índices que se incrementa con acarreo de derecha a izquierda, construyendo la cadena candidata, calculando su hash y comparándolo contra el objetivo hasta encontrar coincidencia.
- **Ataque por Diccionario:** Lee secuencialmente el archivo diccionario.txt línea por línea. Limpia saltos de línea, calcula el hash de cada palabra y lo compara contra el buscado hasta hallarlo o agotar el archivo.

Módulo de Backtracking

- **Backtracking con Poda:** Aplica una búsqueda recursiva en profundidad manteniendo en tiempo real el conteo de cada categoría de carácter.

Poda por Espacio Insuficiente: Si los caracteres faltantes para cumplir la política superan los espacios vacíos restantes, realiza un retorno inmediato e incrementa el contador de nodos podados.

Poda por Repetición Consecutiva: Si el siguiente carácter a evaluar es idéntico al anterior, ignora la rama e incrementa el conteo de poda.

- **Backtracking sin Poda:** Genera todas las combinaciones posibles de longitud n sobre los 67 caracteres y valida la política completa únicamente al llegar a las hojas del árbol.

Módulo de Programación Dinámica

Resuelve la intratabilidad de Backtracking para n mayor o igual que 8 al contabilizar combinaciones por estados en lugar de construir texto carácter por carácter. Registra la posición actual, el último carácter usado y la cantidad acumulada de cada categoría. Al colapsar cadenas equivalentes en una tabla, calcula la cantidad exacta de contraseñas válidas para $n = 8$ y $n = 10$ en menos de un minuto, sirviendo como herramienta de verificación teórica para Backtracking en $n = 6$.

6. Pseudocodigos.

- **Pseudocodigo de Fuerza Bruta**

buscar_fuerza_bruta(hash_objetivo, alfabeto, longitud):

indices = arreglo_de_ceros(longitud)

base = longitud(alfabeto)

para c desde 0 hasta $(base \wedge longitud) - 1$:

 candidato = construir_cadena(alfabeto, indices)

 si calcular_sha256(candidato) == hash_objetivo:

 retornar (verdadero, candidato)

// Incremento con acarreo

para i desde longitud - 1 hasta 0 con paso -1:

 indices[i] = indices[i] + 1

 si indices[i] < base: romper

 indices[i] = 0

retornar (falso, "")

- **Pseudocodigo de ataque por diccionario.**

buscar_diccionario(hash_objetivo, ruta_diccionario):

mientras haya_lineas(ruta_diccionario):

 palabra = limpiar(leer_linea())

 si calcular_sha256(palabra) == hash_objetivo:

 retornar (verdadero, palabra)

retornar (falso, "")

- **Pseudocodigo Backtracking con poda.**

```
backtrack_fast(cadena, low, up, dig, sym, politica):
```

```
    k = longitud(cadena)
```

```
    si k == politica.longitud:
```

```
        si cumple_minimos(low, up, dig, sym): soluciones++
```

```
        retornar
```

```
    // Poda por insuficiencia de espacio residual
```

```
    faltantes = req(low) + req(up) + req(dig) + req(sym)
```

```
    si faltantes > (politica.longitud - k):
```

```
        nodos_podados++
```

```
        retornar
```

```
    para c en alfabeto:
```

```
        // Poda por repetición consecutiva
```

```
        si sin_consecutivos y k > 0 y c == cadena.ultimo():
```

```
            nodos_podados++
```

```
            continuar
```

```
    cadena.agregar(c)
```

```
    backtrack_fast(cadena, low + es_low(c), up + es_up(c), dig + es_dig(c), sym +  
es_sym(c), politica)
```

```
    cadena.eliminar_ultimo()
```

- **Backtracking sin poda**

```
backtrack_without_pruning(cadena, politica):
```

```
    nodos_generados++
```

```
    si longitud(cadena) == politica.longitud:
```

```
si es_valida_completa(cadena, politica): soluciones++  
retornar
```

```
para c en alfabeto:
```

```
    cadena.agregar(c)  
    backtrack_without_pruning(cadena, politica)  
    cadena.eliminar_ultimo()
```

- **Programación Dinámica**

```
contar_soluciones_dp(pos, ultimo_caracter, low, up, dig, sym):
```

```
    si pos == politica.longitud:
```

```
        retornar 1 si cumple_minimos(low, up, dig, sym) sino 0
```

```
    si memo[pos][ultimo_caracter][low][up][dig][sym] != -1:
```

```
        retornar memo[pos][ultimo_caracter][low][up][dig][sym]
```

```
total = 0
```

```
para c en alfabeto:
```

```
    si sin_consecutivos y pos > 0 y c == ultimo_caracter: continuar
```

```
    total += contar_soluciones_dp(pos + 1, c, nuevo_low(c), nuevo_up(c),  
nuevo_dig(c), nuevo_sym(c))
```

```
memo[pos][ultimo_caracter][low][up][dig][sym] = total
```

```
retornar total
```

7. Implementación

El código fue escrito en C++ y compilado con -O2 de g++. Se utilizó la librería <chrono> para medir el tiempo en milisegundos con alta precisión y la librería picosha2.h para calcular los hashes SHA-256.

La estructura del código en la carpeta src/ se dividió en tres módulos principales:

- fb_ejercicio.cpp: Lógica de Fuerza Bruta y lectura de diccionario.
- bt_solver.cpp: Lógica recursiva de Backtracking y función de poda.
- bt_dp.cpp: Algoritmo de Programación Dinámica para contar soluciones exactas en milisegundos.

8. Análisis de complejidad teórica.

Modulo	Complejidad Temporal en el peor caso	Complejidad Espacial	Descripción
Fuerza Bruta	$O(\Sigma ^n)$	$O(n)$	Revisa absolutamente todas las combinaciones posibles.
Ataque por diccionario	$O(M)$	$O(1)$	Solo revisa las M palabras que están en el archivo.
Backtracking con poda	$O(b^n)$	$O(n)$	Reduce la cantidad de ramas a explorar en el árbol.
Backtracking sin poda	$O(b^n)$	$O(n)$	Explora todo el árbol probando combinaciones.
Programación dinámica	$O(n \cdot \Sigma)$	$O(n)$	Cuenta soluciones válidas sumando estados sin generar las cadenas.

9. Casos de Prueba

Semilla asignada: 2472 (obtenida de la función sobre cifuentesmartinezposada).

Instancia de Referencia: Contraseña abc12 (Hash conocido).

Instancias sintéticas probadas en Fuerza Bruta:

- Instancia 1 (n = 4, A1): psrk
- Instancia 2 (n = 4, A2): bcl6
- Instancia 3 (n = 5, A1): jcnkd
- Instancia 4 (n = 5, A2): shi3o
- Instancia 5 (n = 6, A1): ludkpg

10. Experimentación

Entorno de Pruebas

Todas las pruebas de rendimiento y recolección de datos se llevaron a cabo bajo las mismas condiciones de hardware y software para garantizar la consistencia de las mediciones:

- **Procesador:** Intel Core i3
- **Memoria RAM:** 8 GB
- **Sistema Operativo:** Windows 11

11. Metodología de Medición

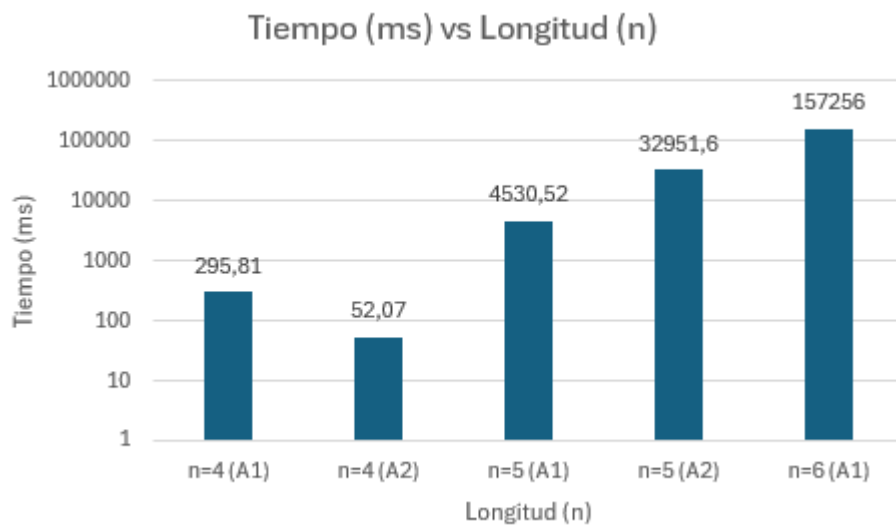
Las mediciones reportadas corresponden a una ejecución única realizada de manera directa sobre el ejecutable binario. Se utilizó la librería estándar <chrono> para tomar el tiempo exacto en milisegundos consumido únicamente por la búsqueda del algoritmo:

- **Módulo Fuerza Bruta:** Se ejecutó el programa en modo --modo fb, evaluando las 5 contraseñas sintéticas generadas por el LCG de la semilla 2472. Los resultados crudos de tiempo y candidatos evaluados se exportaron automáticamente al archivo results/fb_resultados.csv. En paralelo, se realizó la lectura secuencial del archivo resources/diccionario.txt para medir la efectividad del ataque por diccionario.
- **Módulo Backtracking:** Se ejecutó el programa en modo --modo bt, registrando la cantidad de nodos generados, nodos visitados, nodos podados y el tiempo en milisegundos para las distintas variantes de la política.

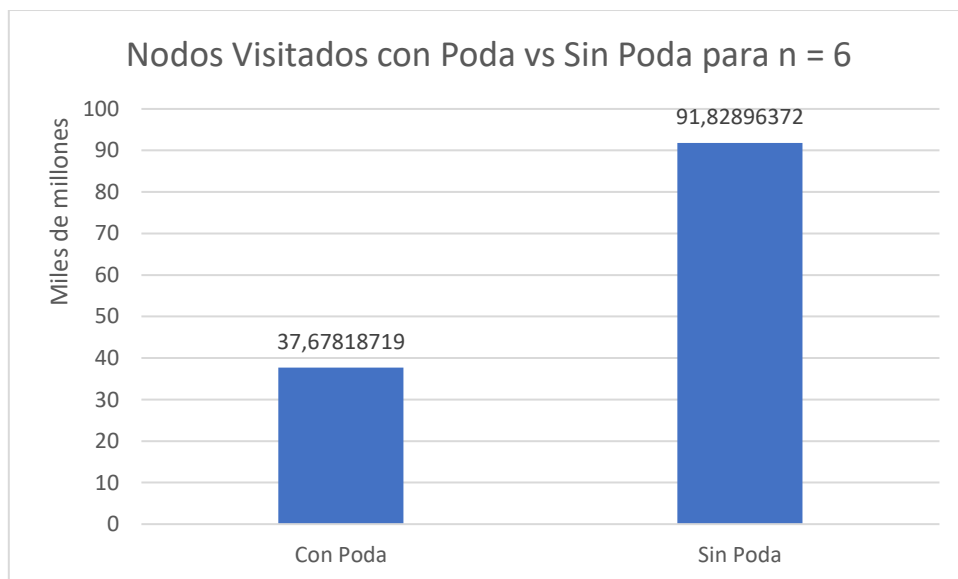
Soportes Visuales Registrados

Las imágenes de los gráficos fueron exportadas y guardadas en la carpeta results/ para su inclusión en el informe:

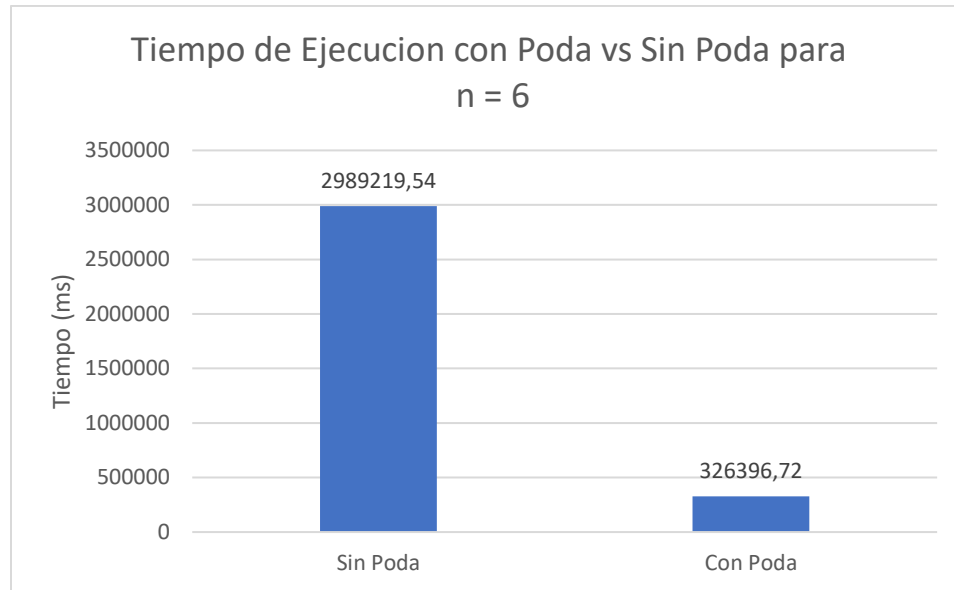
- **Gráfica Fuerza Bruta. Tiempo vs Longitud**



- **Gráfica Backtracking. Nodos Visitados Con Poda vs. Sin Poda para n=6**



- **Gráfica Backtracking. Tiempo de ejecución Con Poda vs. Sin Poda para n=6**



12. Resultados.

Modulo Fuerza Bruta vs Diccionario

Instancia	Longitud (n)	Alfabeto	Contraseña Encontrada	Candidatos Evaluados (FB)	Tiempo FB (ms)	Resultado Diccionario	Tiempo Dict. (ms)
1	4	A1	psrk	276.261	295,81	No encontrada	45,72
2	4	A2	bcl6	49.677	52,07	No encontrada	2,1
3	5	A1	jcnkd	4.156.988	4.530,52	No encontrada	1,72
4	5	A2	shi3o	30.571.107	32.951,6	No encontrada	1,75
5	6	A1	ludkpg	139.894.541	157.256	No encontrada	2,21

Modulo Backtracking.

Variante	Longitud (n)	Soluciones Exactas (DP/BT)	Nodos generados (sin poda/con poda)	Tiempo (Sin / Con Poda)	Reducción del espacio (%)	Speedup por hora
Referencia	6	8.883.856.800	91,82 B / 37,67 B	49,49 min / 5,44 min	58,97%	x9,16
Pol. Completa	8	93.060.290.278.800	$\sim 4,06 \times 10^{14}$ / Intratable	Intratable / Intratable	N/A	N/A
Misma Pol.	6	8.883.856.800	91,82 B / 37,67 B	49,49 min / 5, 44 min	58,97%	x9,16
Misma Pol.	10	$\sim 8,24 \times 10^{17}$	$\sim 1,82 \times 10^{18}$ / Intratable	Intratable / Intratable	N/A	N/A
Relajada	8	$\sim 2,72 \times 10^{14}$	$\sim 4,06 \times 10^{14}$ / Intratable	Intratable / Intratable	N/A	N/A
Poda Nula	6	90.458.382.169	91,82 B / 37,67 B	49,49 min / 49,49 min	0%	x1,00

13. Análisis de resultados.

Contraste Teórico vs Empírico en fuerza bruta.

En el Alfabeto A1, el tiempo crece en proporciones consistentes con $|\Sigma|$:

- De $n = 4$ a $n = 5$ el tiempo sube de 295,81 ms a 4.530,52 ms (x 15,31)
- De $n = 5$ a $n = 6$ el tiempo sube a 157.256 ms (x 34,71)

En el equipo que se hicieron las pruebas $n = 6$ marca el muro exponencial (2,6 minutos por clave.) Lo que para el alfabeto A1 se proyecta que $n = 7$ tomaría aproximadamente 1,13 horas y para $n = 8$ requeriría aproximadamente 29,5 horas.

Efectividad de la poda en backtracking ($n = 6$)

En la instancia $n = 6$, la versión sin poda explora 91.828.963.717 nodos en 49,82 minutos. Al activar la factibilidad:

- Los nodos visitados bajan a 37.678.187.186 podando el 58,97% del árbol
- El tiempo se reduce a 5,44 minutos logrando una aceleración de x 9,16.

Muro exponencial en $n \geq 8$ y densidad de soluciones.

Para $n = 8$, el conteo en backtracking resulta intratable ya que demoraría varios días. Esto es debido a que existen 93.06 billones de soluciones validas bajo la

política. Como Backtracking recorre recursivamente cada hoja válida para contarla, la densidad de soluciones satura la ejecución, demostrando que la poda reduce la constante de tiempo, pero no altera la complejidad exponencial.

14. Comparación Algorítmica.

- **Fuerza Bruta vs Diccionario:** El diccionario responde en un tiempo menor que 45,73 ms, pero obtuvo un 0% de éxito ante las claves sintéticas de la semilla. Lo que muestra que es demasiado rápido pero incompleto, mientras que el algoritmo de fuerza bruta es completo pero costoso exponencialmente.
- **Backtracking con poda vs sin poda:** La poda recorta ramas no factibles en niveles tempranos, descartando el 58,97% del árbol.
- **Backtracking vs Programación Dinámica:** Backtracking es la técnica para generar cadenas. Para contar en espacios que igualen o superen a $n = 8$, calculando el total exacto en poco tiempo agrupadamente por estados, superando los varios días que demoraría backtracking.

15. Conclusiones

- La incorporación de funciones de factibilidad anticipadas en Backtracking reduce significativamente el espacio de exploración en árboles de decisión, logrando omitir el 58,97% de los nodos e incrementando la velocidad en $\times 9,16$ para $n = 6$.
- El algoritmo de Fuerza Bruta pura sobre funciones hash como SHA-256 presenta una barrera infranqueable de tratabilidad práctica en $n = 6$ para procesadores personales convencionales, confirmando la cota asintótica $O(|\Sigma|^n)$.
- Para la verificación de políticas de contraseñas con alta holgura en $n \geq 8$, la densidad masiva de soluciones válidas (93,06 billones) inutiliza el conteo mediante Backtracking por enumeración de hojas, haciendo indispensable el uso de modelos de Programación Dinámica para el análisis combinatorio exacto.
- Los ataques por diccionario garantizan respuestas en pocos milisegundos, pero fallan rotundamente, por ejemplo, en nuestro caso con un 0% de efectividad, ante contraseñas sintéticas, demostrando que la seguridad criptográfica no puede ser vulnerada mediante listas finitas no exhaustivas.

16. Referencias Bibliográficas.

- Cormen, T. H. et al. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

- Universidad EAFIT. (2026). *Guía de la Práctica 1: Fuerza Bruta y Backtracking*. Análisis y Diseño de Algoritmos.
- Okvis. (2026). Picosha2: A C++17 header-only SHA256 library. GitHub: <https://github.com/okvis/picosha2>

17. Declaración de Transparencia Ética sobre el Uso de IA

Herramientas usadas: Google Gemini / ChatGPT / Claude (agosto 2026).

Propósito de uso:

- Asistencia de sintaxis para la elaboración de los módulos de Fuerza bruta, backtracking y programación dinámica.
- Asistencia de redacción del informe técnico.
- Asistencia para la formulación de estados de la programación dinámica.
- Asistencia con la revisión de parámetros cumplidos y faltantes en la elaboración del GitHub e informe técnico.
- Elaboración del README.md

Todo el código fuente fue implementado y verificado por el equipo. Las métricas reportadas provienen de ejecuciones empíricas reales.

18. Declaración de Contribución Individual

- **Alejandro Cifuentes Arroyave:** Módulo Fuerza Bruta, integración de picosha2.h y exportación de datos a CSV.
- **Juan Daniel Martínez Fernández:** Módulo Backtracking y desarrollo de la función de poda.
- **Samuel Posada Londoño:** Módulo de Programación Dinámica, recolección de métricas experimentales y redacción del Informe Técnico.